

CSE 306

Assignment-2: 32-bit Floating Point Adder Simulation

Section – B2

Group - 03

Members:

1. 2005095 - Tamim Hasan Saad
2. 2005096 - Habiba Rafique Saima
3. 2005111 - Sadnam Faiyaz
4. 2005114 - Shahriar Kabir
5. 2005119 - Sadia Afrin Sithi

1. Introduction

Floating point numbers are real numbers represented by computers that have a fixed number of digits before and after the decimal point. A floating-point adder is a type of digital circuit used to add and subtract floating point integers. It can be used for numerical values that are too large or too little for integer representations to fully capture. The precision provided by floating point arithmetic is essential in fields like scientific computing, financial modeling, and machine learning, where accurate and flexible representation of real-world values is paramount for reliable computational outcomes.

Three fields make up a number's floating-point representation: the sign bit, the exponent field, and the mantissa field. The positive or negative sign is represented by the sign bit. To represent a wide range of values, the exponent field enables the significant to be multiplied by a power of the base using a set number of bits in a biased form. The bias needs to be deducted from the stored bits in order to get the real exponent. The fractional portion of the number, or mantissa, is made up of the digits that come after the decimal point. The exponent field, the mantissa, and the sign bit use 1, 11, and 20 bits in this implementation, respectively. Thus, the exponent bias for our problem would be $212-1-1 = 2047$.

In order to add two numbers, a floating-point adder first aligns their decimal points before adding their mantissas. Once the necessary changes (increment/decrement) have been made, the result's exponent is determined. Normalization and rounding are then performed. The signs of the two integers being added determine the sign of the outcome. This meticulous process ensures that precision is maintained, and the final result conforms to the standardized representation of floating-point numbers, facilitating accurate numerical computations in various computational domains.

Financial modeling, computer graphics, and scientific and engineering computations are just a few of the many uses for floating point adders. Because floating point arithmetic can be computationally demanding, co-processors use it to execute quick, hardware-accelerated operations. The main CPU cannot complete these calculations nearly as quickly as a dedicated floating-point adder. Applications like data analysis, scientific simulation, and other jobs requiring high-precision floating point calculations may find this to be particularly crucial.

2. Problem Specification

The assignment asks to design a floating-point adder circuit which takes two floating points as inputs and provides their sum, another floating point as output. Each floating point will be 32 bits long with the following representation:

Sign	Exponent	Fraction
1 Bit	11 Bits	20 Bits

3. Description and Circuit Diagram of Modules

Several libraries have been implemented for the sake of modularity in the design of the floating-point adder. Descriptions and usages of the libraries are given below:

3.1 Multiplexer Library

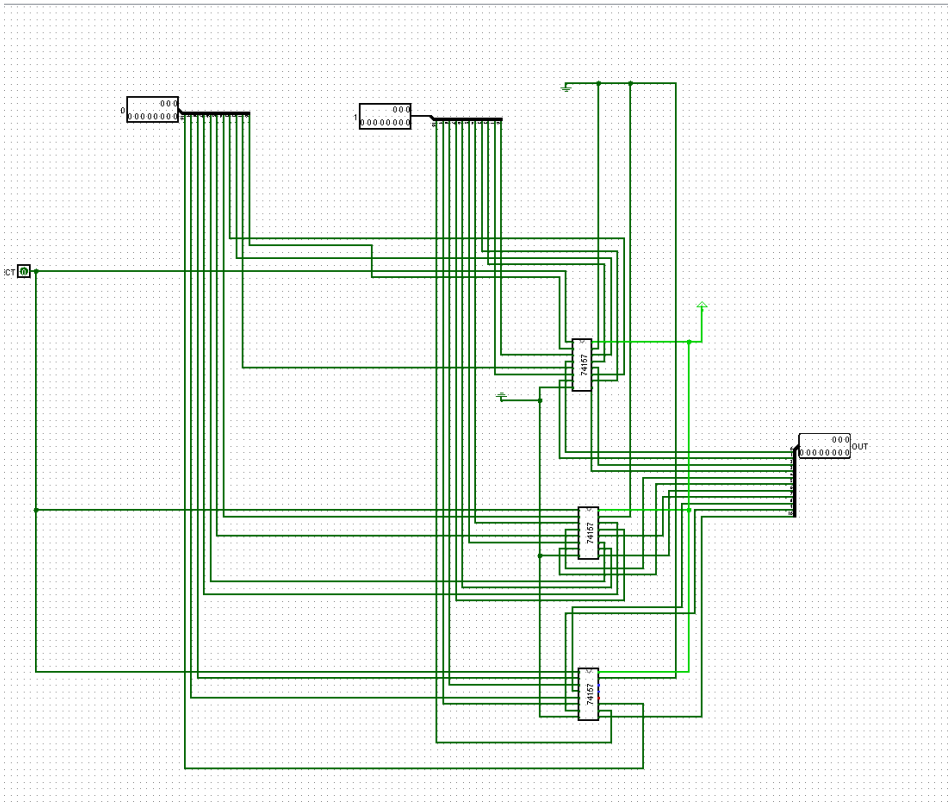


Fig: MUX_11_bit

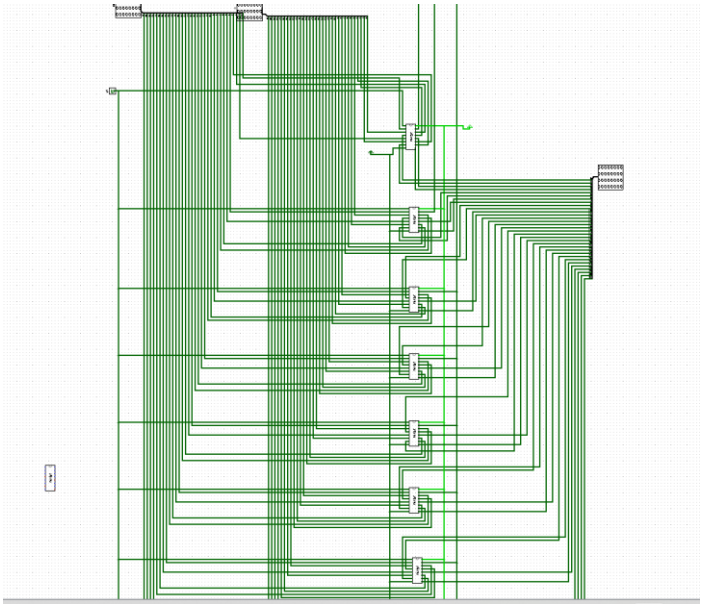


Fig:MUX_32_bit

3.2 Comparator Library

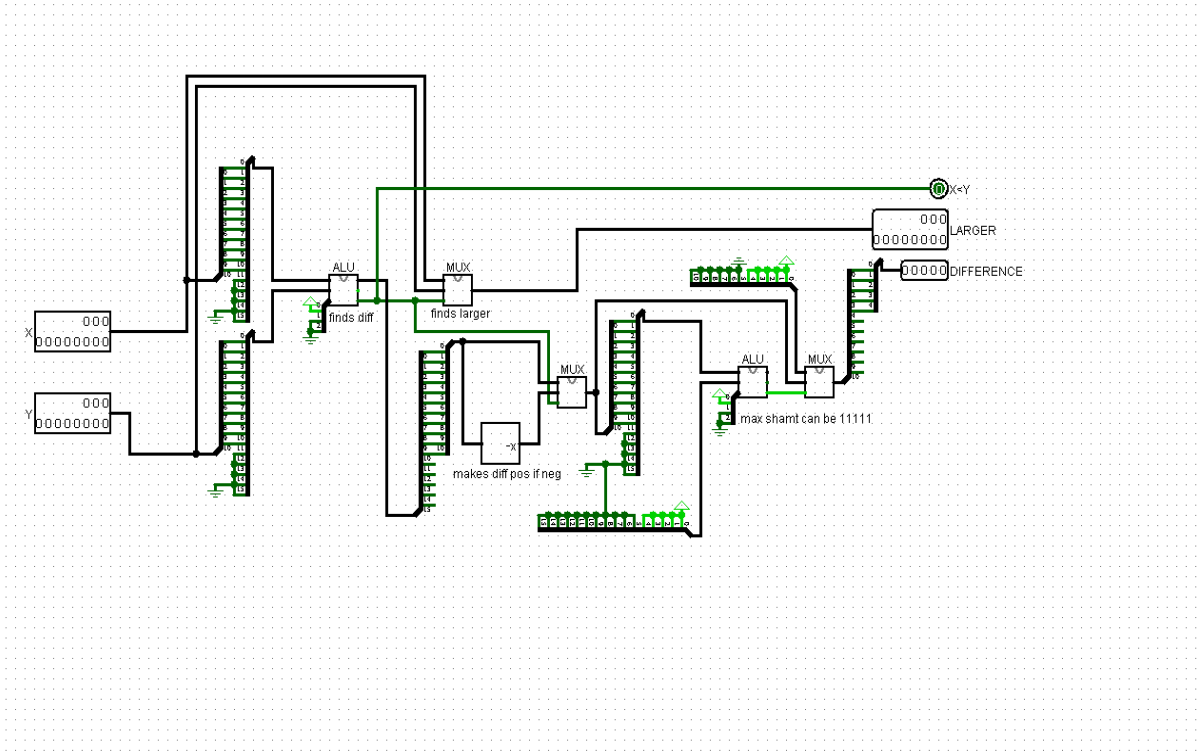
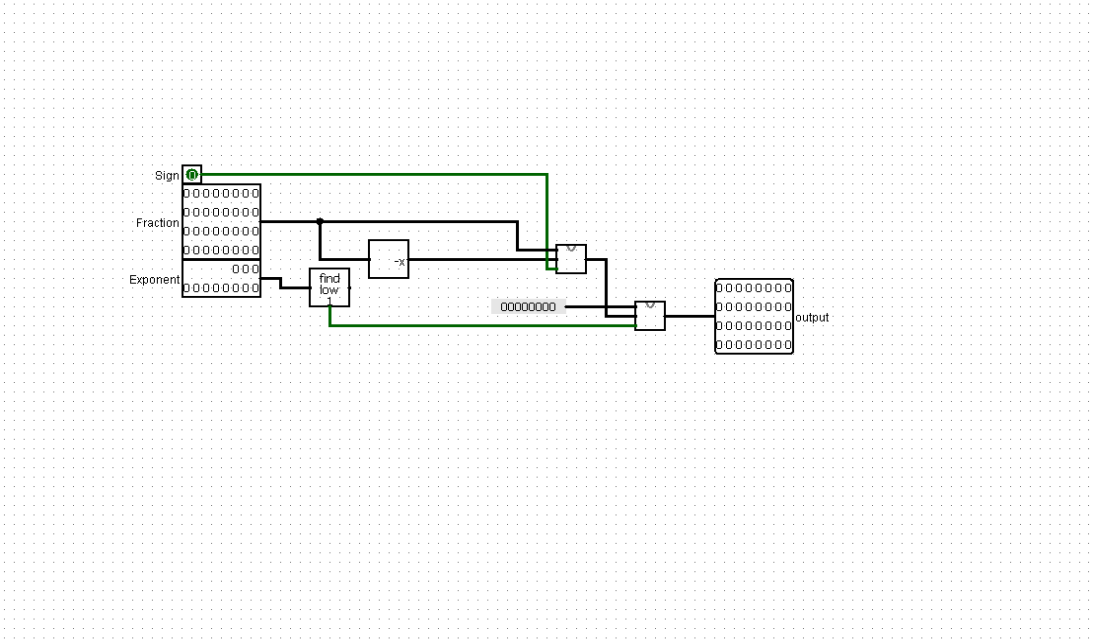


Fig: Comparator

3.3 Sign Check Library



3.4 Normalizer Library

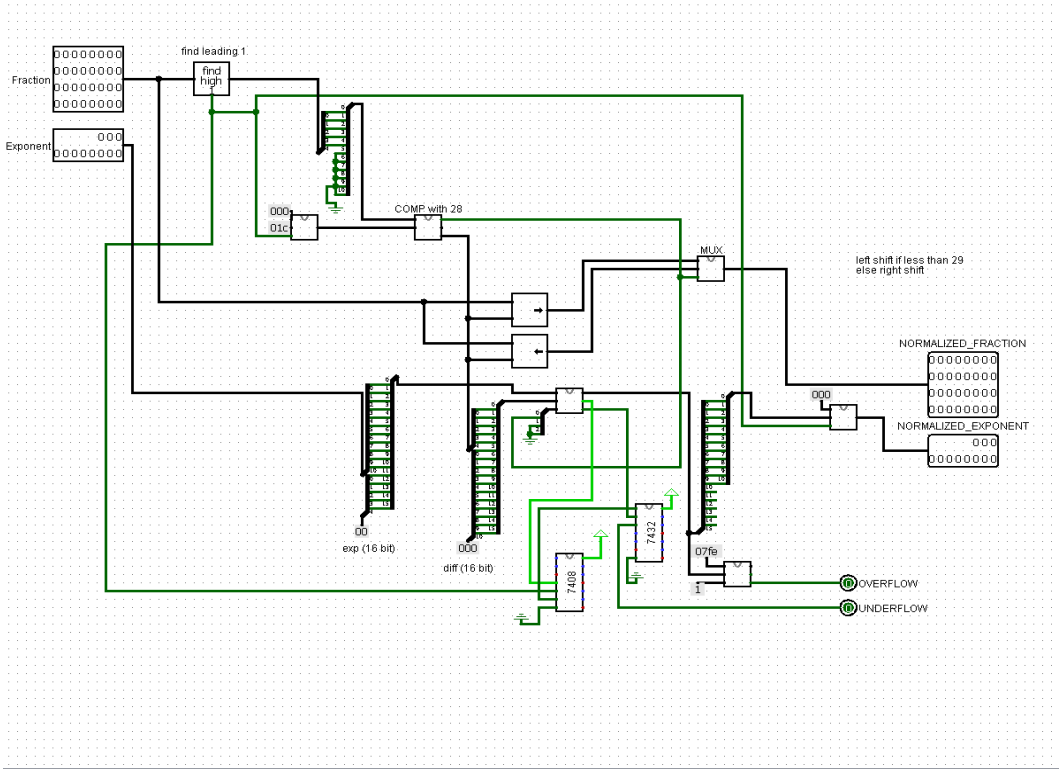


Fig: Normalizer

3.5 Rounding Circuit

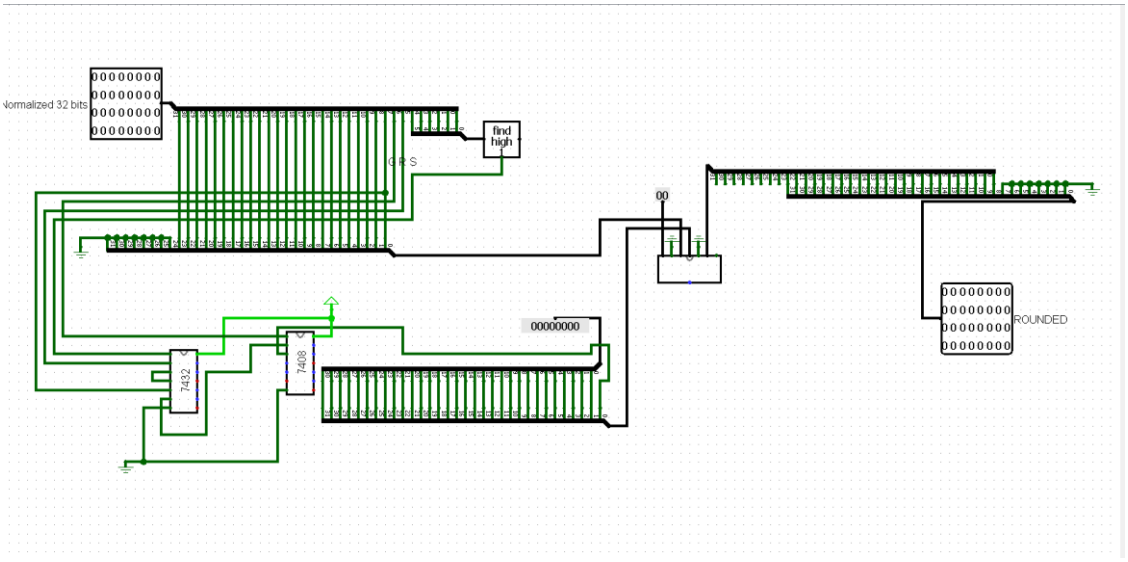


Fig: Rounder

3.6 Floating Point Adder

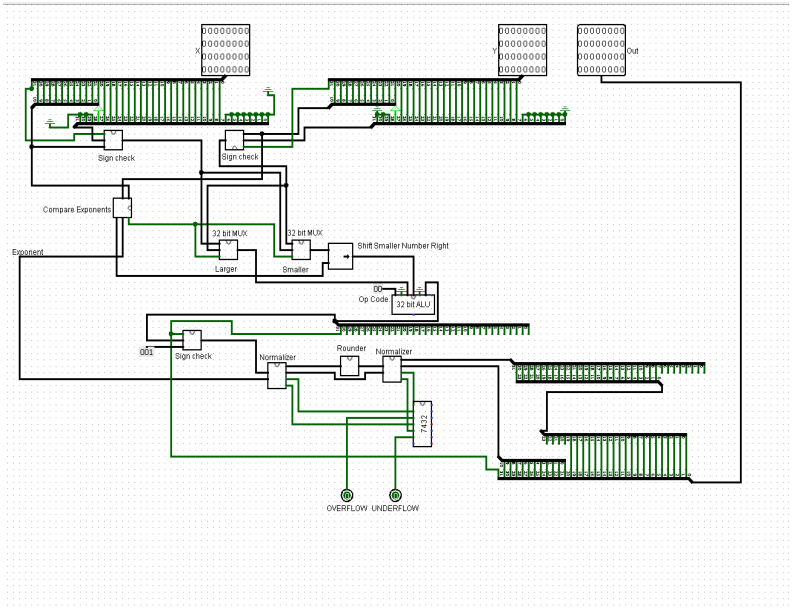


Fig: FPA

3.7 Adder Module

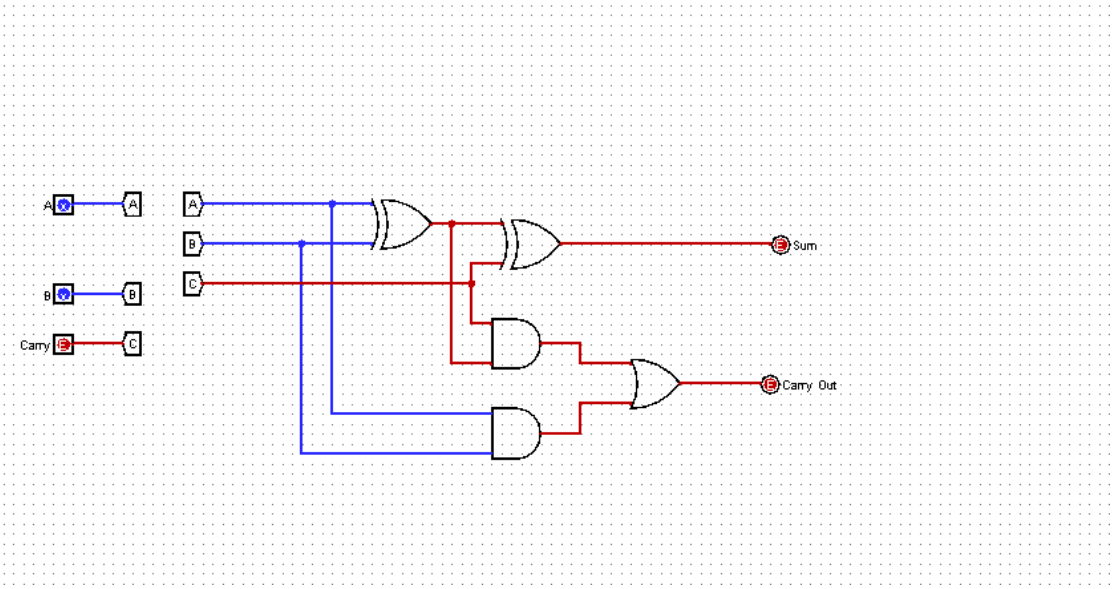


Fig: 1 bit Adder

3.8 Main Circuit

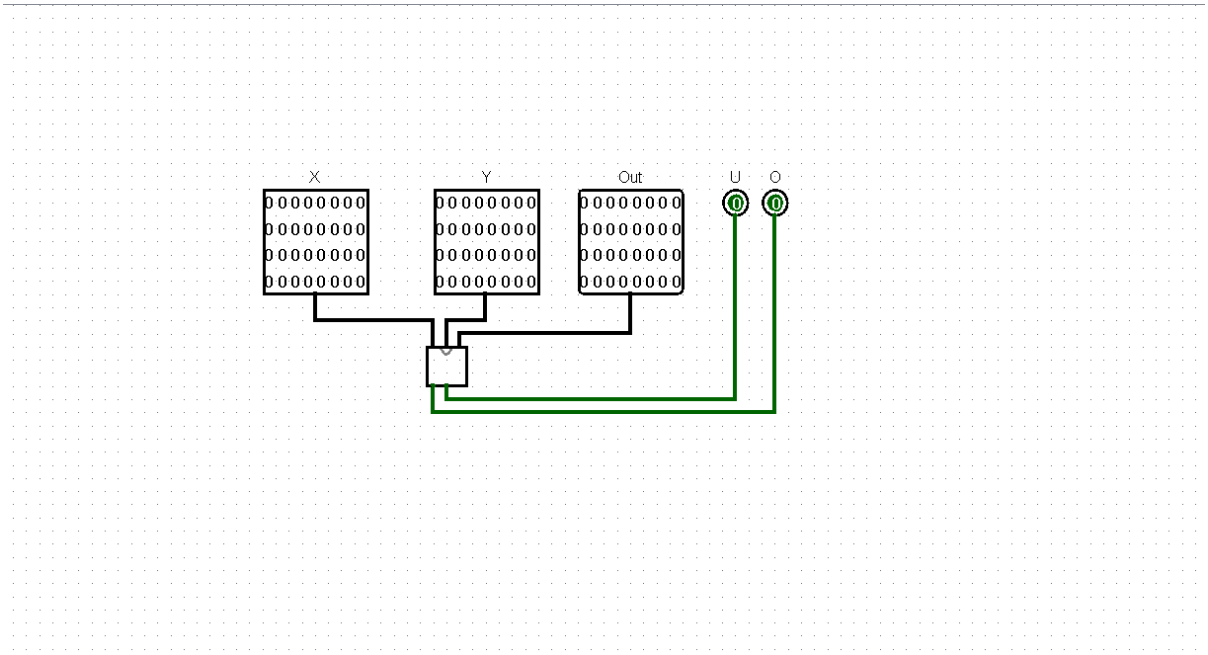


Fig: Main Circuit

4. Flowchart

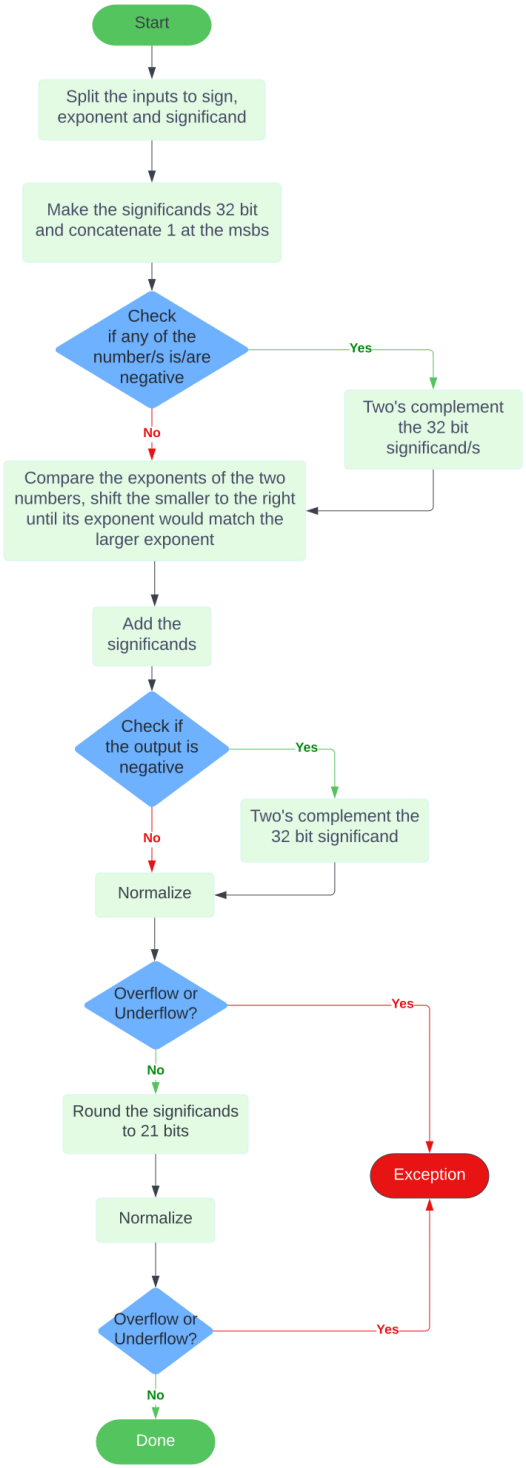


Fig: Flow chart

5. High-level Block Diagram of the Architecture

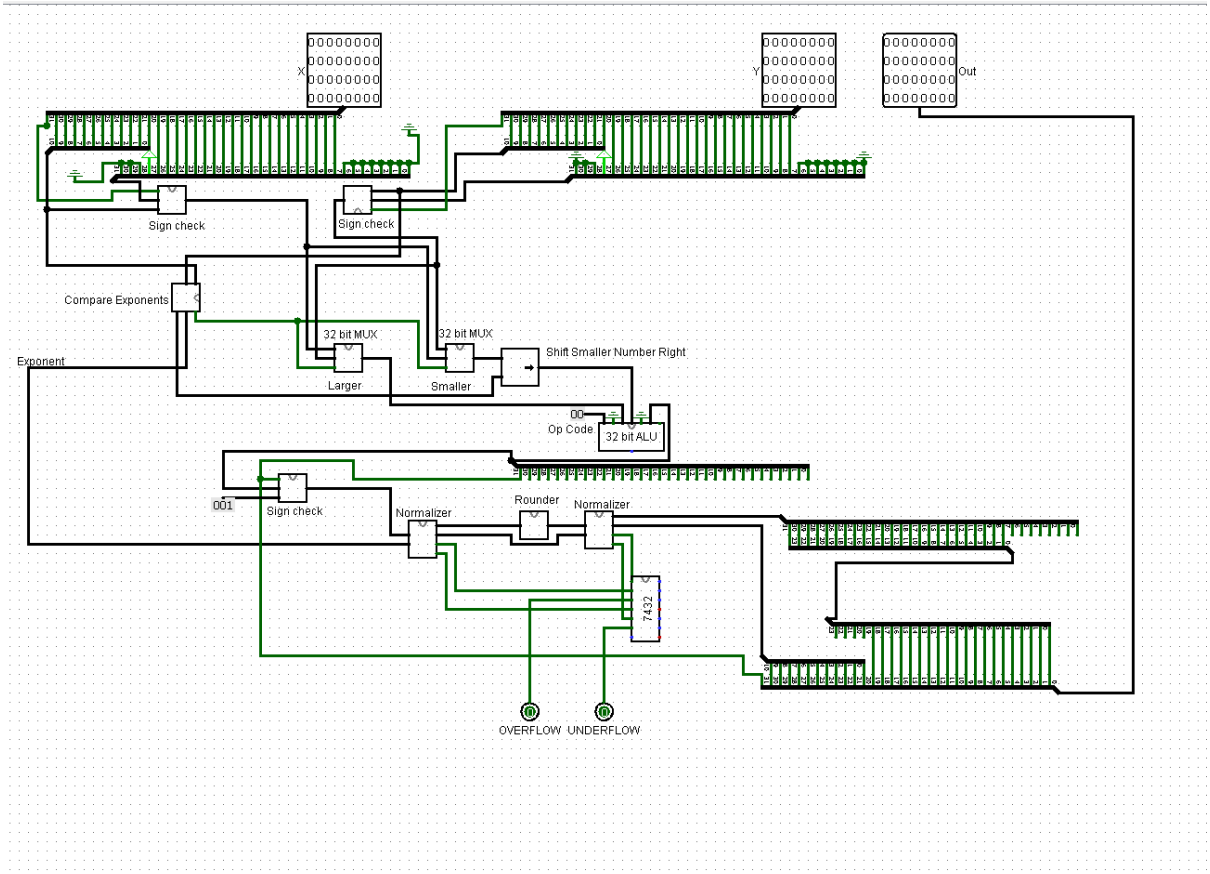


Figure 16: Block Diagram of FPA

6. Comprehensive Design Description

6.1 Comparing the Exponents and Aligning Radix Point

The radix points need to be lined up in order to add two floating-point values. To align the input with the larger input, this is usually accomplished by moving the input with the smaller exponent to the right. A 12-bit subtractor was utilized to determine the difference between the two inputs' exponents after the exponents were compared using a comparator.

6.2 Shifting

All shifting operations have been carried out using shifters based on multiplexers. Through the employment of five 32-bit multiplexers, bit shifts of any desired length up to 31 have been obtained. The multipliers have a 1, 2, 4, 8 and 16 bit shift capacity, respectively. As demonstrated in 4f and 4l, by combining them, we may obtain any number of shifts up to 31. We might require more than 31 shifts to align two fractions, in which case all bits would be 0. Furthermore, the exponent difference ought to be the shifter circuit's input. Consequently, when all seven of the higher bits are clear, the lower five bits are utilized for shifting. The shift amount does not surpass 31 ($2^5 - 1$) unless such is the case. Otherwise, the fraction will be 0(all bits cleared), which is done in 4m.

6.3 Normalization

We used a priority encoder to determine the location of the most important set bit, which is the leftmost one, in order to normalize and then do an addition operation. To put this into practice, four 8 to 1 priority encoders are used. The outcome can range from 0 to 31 (or 31 to 0), meaning it will consist of 5 bits ($0 – 2^5 - 1$). The result's lower three bits are the encoders' output. The valid bits of the encoders are used to choose the upper two bits and outputs. Other encoders won't be chosen and the top two bits will be 00 if the leftmost encoder receives a valid input. Therefore, the decimal range of the integers that are formed is 0 to 7. Likewise, if the second encoder is chosen, the top two bits will be 01; the third and fourth ones will be 10 and 11, respectively. A 4 to 2 priority encoder, whose inputs are the valid bits of the previously stated priority encoders, passes the selection bit of the multiplexer. In the event that none of the priority encoders accepts an input, the fraction is made up entirely of zeros. Two scenarios can result in this.

1. The result is of the structure 1.00×2^x
2. The result is 0.0

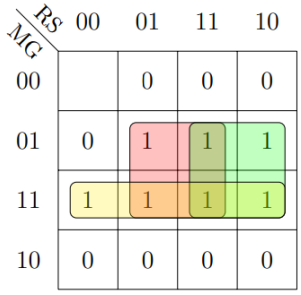
The first case can appear if two numbers of type $1.00 * 2$ x−1 and equal exponents are added. So, we can check whether the signs of both inputs are same. If they are, we will continue the algorithm and increase the exponent by 1. If the signs are opposite, the result is actually 0. So, output is 0.0.

6.4 Rounding

In order to add and subtract the mantissas, we employed a 32-bit adder. However, we can only retain 20 bits, so we may need to round the result in some instances. We consider 21th bit as Guard bit, 22th as round bit. If any of the bits on the right of round bit is set, Sticky bit will be set. Otherwise, it is clear. We need to consider these cases:

^{20th} bit	G	R	S	Action
X	0	X	X	Truncate
1	1	X	X	Round up
0	1	0	0	Truncate
X	1	1	X	Round up
X	1	X	1	Round up

Actually, X100 is the case for round to even. So, if 21th bit is 0, we need to do nothing, hence truncation. Otherwise, we will round up. For simplification we will use K-map. (20thbit = M)



$$flag = GS + GR + MG = G(M+R+S)$$

If flag is set, 1 is added at the 20th position. Otherwise, the bits beyond the 21th position are truncated.

6.5 Computing the Sign Bit

6.5 Computing the Sign Bit There are certain considerations to take into account when calculating the sign bit. If the signs of the two inputs are the same, then the sign of the output will be either of the signs of the inputs. If they are not the same, we have to consider two things. Firstly, we need to find the sign of the output of the adder. It can be calculated using the equation:

$$sign = (S_A \oplus S_B)\overline{C_{out}} \quad (1)$$

Where Cout is the carry out of the adder. However, this sign may not always be correct because we always subtract the input with the smaller exponent from the input with the higher exponent. When the signs of the inputs are opposite, i.e, if the input with greater exponent is negative and the other one is

positive, 1 gives opposite to the correct. To fix this we have introduced a variable switch. This switch bit will alter the sign bit in those cases. Our sign bit will fail when the sign bits of inputs are opposite and the input with greater exponent is negative. In that case switch will become 1 and will alter the sign. Equation of switch bit is:

$$\text{switch} = (S_A \oplus S_B)(\text{Comp} \oplus S_B) \quad (2)$$

Here, Comp is the output of the comparator circuit ($Exp_A > Exp_B$). So, the formula for sign bit:

$$\text{actualSign} = \text{sign} \oplus \text{switch} \quad (3)$$

Equation 3 will be used in case the signs of the inputs are different. Otherwise, The sign of the first input will be directly applied to the sign of the output. This selection will be made using a multiplexer.

6.6 Handling Zero or Small Input

If either of the inputs is zero, the other input will be passed directly to the output. Additionally, if the exponent of one of the inputs is significantly smaller than the other (a difference of more than 31), the input with the larger exponent is passed directly to the output.

6.7 Increasing Precision

For addition, we did not set aside a bit to capture the overflow bit. Instead, we captured the bit directly from the carry out. Besides, in case of subtraction, we did not use a separate bit for the sign. Instead, the carry out of the adder was used to determine the sign of the result. If the carry out is 1, the result of subtraction is positive, otherwise negative. It is an exception when one of the summands is zero. Therefore, this case was handled separately as stated in 6.6. As a result, the precision is increased by 2.

7. ICs Used with Count as a Chart

IC	Quantity
74157	55
7432	3
7408	2
7483	10

8. Simulator used Along with the Version Number

Logisim 2.7.1 has been used for simulating the floating-point adder circuit.

9. Discussion

We worked hard on this Assignment to make sure the design was unique and as efficient as possible. To avoid utilizing extra bits, we often went the challenging path and implemented the modules ourselves or invented an algorithm. Even though it took a lot of work, we were able to produce a design that is as unique as it gets.

We didn't set aside a different bit to record overflow or the result's sign, as was mentioned in 6.7. As a result, we were able to increase our implementation's precision by two numbers. But developing the right algorithm to compute the result accurately required some time. To get over the restriction of having no additional bit, we used the carry out from the adder.

Apart from the technical components of our solution, we also made sure that our floating-point adder was self-contained. That is to say, we implemented every modular part of our adder ourselves, without using any third-party libraries. This improved our tool's dependability and ease of maintenance in addition to enabling us to completely comprehend its internal workings. We accomplished everything, from utilizing multiplexers to shift bits to employing priority encoders to locate the leftmost set bit.

All things considered, designing and implementing the floating point adder was an exciting assignment that taught us a lot of new information.